

Relazione - Versione 9: Multi-Step Agent

Introduzione

La Versione 9 introduce il Multi-Step Agent. Nella Versione 8 il sistema era già in grado di effettuare routing semantico e parsing semantico, ma continuava a eseguire una sola azione per richiesta. La Versione 9 mantiene gli stessi tool, il Tool Registry, l'LLM Router e l'LLM Parser, e aggiunge una semplice forma di esecuzione sequenziale.

Il Planner ora crea una lista ordinata di step e l'Agent esegue questi step uno alla volta. Il risultato di uno step può essere riutilizzato nello step successivo.

Obiettivo della Versione 9

L'obiettivo principale della Versione 9 è introdurre l'esecuzione multi-step mantenendo l'architettura facile da comprendere.

- Il Planner crea una sequenza ordinata di richieste di step.
- L'Agent esegue ogni step in modo sequenziale.
- Il risultato di uno step può essere inserito nello step successivo.
- LLM Router e LLM Parser rimangono invariati.
- Il Memory Manager salva l'intera interazione multi-step.

Cosa cambia rispetto alla Versione 8

La Versione 8 aveva già un LLM Router e un LLM Parser, ma il Planner restituiva un solo piano e l'Agent eseguiva una sola azione. La Versione 9 modifica il Planner in modo che restituisca una lista di elementi del piano. Modifica inoltre l'Agent affinché iteri su questi elementi e tenga traccia dei risultati intermedi.

Nuovo concetto: pianificazione multi-step

Il Planner ora divide le richieste che contengono istruzioni sequenziali separate da "then". Ogni sotto-richiesta viene instradata in modo indipendente e trasformata in uno step del piano.

```
def split_request(request):
    return [
        step.strip(" ,.")
        for step in request.split(" then ")
        if step.strip()
    ]
```

Per la richiesta di esempio qui sotto, il piano contiene due step ordinati:

```
agent("Add 2 and 3, then multiply the result by 4.")

[
    {"request": "Add 2 and 3", "tool": "addition", ...},
    {"request": "multiply the result by 4", "tool": "multiplication", ...}
]
```

Passaggio del risultato allo step successivo

La Versione 9 aggiunge una piccola funzione di supporto che risolve i riferimenti al risultato precedente. L'implementazione rimane volutamente semplice e didattica.

```
def resolve_step_request(request, previous_result):
    if previous_result is None:
        return request
    return request.replace("the result", str(previous_result))
```

Dopo che lo Step 1 calcola 5, lo Step 2 riceve la richiesta "multiply 5 by 4" e l'LLM Parser esistente può estrarre normalmente gli argomenti.

Architettura completa nella Versione 9

Il flusso di esecuzione diventa:

```
Request -> Planner -> Step 1 (LLM Router -> LLM Parser -> Executor)
                        -> Step 2 (LLM Router -> LLM Parser -> Executor)
                        -> ...
                        -> Memory Manager -> Memory
                              |
                              v
                        Output
```

Stato dell'agente nella Versione 9

Lo stato ora memorizza una lista di elementi del piano e una lista degli step eseguiti:

```
{
  "request": "Add 2 and 3, then multiply the result by 4.",
  "plan": [
    {"request": "Add 2 and 3", "tool": "addition", ...},
    {"request": "multiply the result by 4", "tool": "multiplication", ...}
  ],
  "steps": [
    {"request": "Add 2 and 3", "action": "addition", "arguments": {"a": 2,
      "b": 3}, "result": 5},
    {"request": "multiply the result by 4", "action": "multiplication",
      "arguments": {"a": 5, "b": 4}, "result": 20}
  ],
  "result": 20
}
```

Test

La Versione 9 mantiene i test principali single-step e aggiunge un nuovo test multi-step:

```
clear_memory()
tool_registry
agent("What is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
agent("Could you calculate the product of 2 and 3?")
agent("Multiply three by four.")
agent("Add 2 and 3, then multiply the result by 4.")
get_memory()
get_last_interaction()
```

- I primi cinque test verificano che la Versione 9 preservi il comportamento single-step della Versione 8.
- Il nuovo test multi-step verifica che il Planner crei due step ordinati.
- Verifica inoltre che il risultato del primo step venga passato al secondo step e diventi parte della richiesta successiva da parsare.

- La memoria ora salva l'intera interazione multi-step, compreso il piano completo e gli step eseguiti.

Cosa insegna la Versione 9

La Versione 9 insegna come un agente possa passare da una sola azione a una piccola sequenza di azioni. Il cambiamento rimane volutamente semplice: il planner divide in base a "then", l'agent esegue step dopo step e il risultato di uno step può essere riutilizzato in quello successivo.

Questa versione è lo stadio finale implementato del progetto. Il sistema è ora in grado di coordinare i tool, salvare i metadati in un Tool Registry, usare un modello locale per routing e parsing ed eseguire piccole richieste multi-step.

Limiti

- la pianificazione multi-step è volutamente semplice e dipende dalla parola "then";
- vengono risolti esplicitamente solo i riferimenti a "the result";
- il piccolo modello locale può ancora commettere errori di routing o parsing;
- il Tool Registry rimane in memoria e la registrazione è manuale;
- la memoria rimane una semplice lista.

Conclusione

La Versione 9 è la conclusione naturale della roadmap basata su notebook. Mantiene l'architettura modulare pulita sviluppata nelle versioni precedenti e aggiunge la capacità pratica finale: l'esecuzione multi-step.

Il test riuscito "Add 2 and 3, then multiply the result by 4." dimostra chiaramente il beneficio principale della versione. L'agente è ora in grado di pianificare una sequenza, eseguirla step dopo step, passare in avanti i risultati intermedi e salvare l'intero stato dell'interazione.